

01-00

Chapter Overview – What You Will Learn

Topic	AWS Services Covered	ShopFlow Use-Case
Global Infrastructure	Regions, AZs, Edge Locations	Choose deployment region
Core Concepts	IAM, VPC, EC2, S3	Foundation for every chapter
ShopFlow Architecture	20+ AWS services wired together	End-to-end e-commerce app
Pricing Model	On-Demand, Reserved, Spot, Free Tier	Cost planning for startup
AWS Console & CLI	Management Console, AWS CLI v2, SDK	Day-to-day operations
Shared Responsibility	AWS vs Customer security split	Compliance baseline
Account Structure	Organizations, SCPs, OUs	Multi-account governance
Well-Architected	6 pillars framework	Architecture review

01-01

What is AWS & Why It Dominates

Amazon Web Services (AWS) is the world's largest cloud platform offering 200+ fully-managed services — compute, storage, databases, networking, AI/ML, analytics, and security — all billed pay-as-you-go. AWS holds ~31% global cloud market share (2024), powering Netflix, Airbnb, NASA, and millions of companies.

Key Value Propositions

- No upfront capital — replace CapEx with OpEx; pay only for what you use, by the second
- Elasticity — scale from 1 server to 10,000 in minutes using Auto Scaling Groups
- Global reach — 33 Regions, 105 AZs, 600+ CloudFront PoPs worldwide (2024)
- Managed services — AWS handles OS patching, backups, failover, hardware replacement
- Security by design — 300+ security, compliance, and governance services
- Broad ecosystem — 15,000+ AWS Marketplace third-party integrations

AWS vs Traditional DC	Traditional Data Center	AWS Cloud
Capacity planning	Buy peak upfront, sit idle 80% of time	Provision in minutes, scale dynamically
Lead time	Weeks to months for new servers	Seconds to minutes via CLI/API
OPEX model	High fixed costs regardless of usage	Variable, pay-per-use pricing
Global reach	Expensive, slow, requires physical presence	Click to deploy worldwide
Redundancy	Manual, costly DR site, rarely tested	Built-in Multi-AZ and Multi-Region
Security patches	Manual, often delayed, risky	Managed service: auto-patched by AWS

AWS Global Infrastructure (2024)

38 Geographic Regions (us-east-1, eu-west-1, ap-south-1...)

105 Availability Zones — isolated data centers per Region

3-6 AZs per Region with independent power, network, cooling

Connected via redundant 100 Gbps private fiber links

600+ Edge Locations (CloudFront CDN / Route 53 DNS)

Content cached within 10ms of end users worldwide

Local Zones — sub-10ms latency near metro areas (NYC, LA, etc.)

Wavelength Zones — AWS compute embedded in 5G carrier networks

AWS Outposts — AWS hardware racks in your on-premises data center

■ ShopFlow deploys to us-east-1 (primary) + eu-west-1 (DR). Choose a region close to your users with 3+ AZs.

01-02

ShopFlow – Reference E-Commerce Application

Every AWS service in this tutorial is demonstrated through ShopFlow — a production-grade e-commerce platform supporting 1M+ monthly active users. Modeled after Amazon/Flipkart, it handles the full retail lifecycle.

Feature	Requirement	AWS Services Used
Product Catalog	500,000+ SKUs, images, recommendations	OpenSearch + DynamoDB + S3
User Accounts	OAuth 2.0, email/pass, 1M+ users	Cognito + Aurora PostgreSQL
Shopping Cart	Persist across sessions, merge guest+user	DynamoDB + ElastiCache Redis
Order Processing	place → pay → fulfil → ship → deliver	Aurora + SQS + Step Functions
Payments	Stripe PCI-DSS, refunds, disputes	Lambda + Secrets Manager
Notifications	Email, SMS, push, WhatsApp	SNS + SES + Pinpoint
Search	Full-text, facets, autocomplete	OpenSearch Service
Analytics	Real-time sales dashboard, funnel	Kinesis + QuickSight
Images & Assets	500GB product images, CDN delivery	S3 + CloudFront
API Layer	REST + GraphQL + WebSocket	API Gateway + AppSync

ShopFlow Traffic & Scale Targets

- Normal load: 1,000 req/s peak, 200 req/s average across all microservices
- Peak events (Black Friday, Diwali Sale): 10x traffic spike → 10,000 req/s sustained
- Order volume: 50,000 orders/day steady-state, 500,000/day peak events
- SLA targets: 99.99% availability (52 min downtime/year), p99 API latency <500ms
- Data volume: 5TB Aurora DB, 500GB DynamoDB, 500GB S3 images, 100GB OpenSearch

ShopFlow – Complete Architecture Overview

Client Layer — React SPA + iOS/Android Mobile Apps

Static assets: S3 + CloudFront CDN (sub-50ms globally)

Real-time inventory: WebSocket via API Gateway + Lambda

Edge & Security — Global

Route 53: DNS failover, health-check-based routing

CloudFront + WAF: CDN + OWASP protection, rate limiting

API & Compute — Private VPC

ALB: HTTPS termination, path-based routing to 5 services

ECS Fargate: product-svc, order-svc, user-svc, notify-svc

Lambda: thumbnails, email render, cron jobs, webhooks

Data — Private Data Subnets

Aurora PostgreSQL Multi-AZ: orders, users, products master DB

DynamoDB: sessions, carts, inventory (sub-ms, auto-scale)

ElastiCache Redis: hot product cache, rate limit counters

OpenSearch: full-text product search + Kibana analytics

Async & Messaging

SQS: order-queue, email-queue, DLQ for failure handling

SNS: order events fan-out to email + SMS + push channels

Kinesis: clickstream (3 shards), inventory-events (2 shards)

An AWS Region is a geographically isolated cluster of data centers. Each Region contains Availability Zones (AZs) — physically separate facilities within ~100km of each other, with independent power, networking, and cooling.

How to Choose a Region — Decision Matrix

Criterion	Weight	Consideration	ShopFlow Decision
User latency	High	Choose closest region to user base	us-east-1 for US, eu-west-1 for EU
Data residency	Critical	GDPR, India PDPA, China regulations	eu-west-1 mandated for EU customers
Service availability	High	Not all 200+ services in every region	Check AWS Regional Services list
Pricing	Medium	Prices vary 5-30% across regions	us-east-1 typically cheapest
Compliance certifications	Critical	HIPAA, PCI-DSS, FedRAMP per region	Verify per compliance program
Direct Connect availability	Medium	On-prem hybrid connectivity	Available in all major Regions

Availability Zone Best Practices

- AZ identifiers: us-east-1a, us-east-1b, us-east-1c (3-6 per Region)
- AZ-to-label mapping is randomized per account to spread load across physical AZs
- Cross-AZ data transfer: \$0.01/GB — minimize with service-mesh or co-location
- ShopFlow spans all 3 AZs: ALB (all 3), ECS tasks (spread), RDS (Multi-AZ 1a+1b)

AZ Failure Scenario	Single AZ	Multi-AZ (2 AZs)	Multi-AZ (3 AZs)
Single AZ down	100% outage	60-120s failover	Zero impact, traffic rerouted
AZ network partition	Data loss risk	Auto recovery	Full redundancy
AZ hardware failure	Hours to recover	Transparent failover	No user impact
Planned maintenance	Downtime window	Rolling restarts	Zero-downtime rolling
AZ-level DDoS	Absorb or fail	Partial mitigation	DDoS isolated to 1 AZ

■ Never run production in a single AZ. Even "high availability" needs at least 2 AZs; target 3 for true fault tolerance without capacity issues.

AWS edge infrastructure serves ShopFlow's static content and accelerates API calls globally. Without CDN, users in Mumbai wait 300ms for images from us-east-1. With CloudFront, they get <10ms from the nearest edge PoP.

Edge Service	Purpose	Latency Impact	ShopFlow Config
CloudFront CDN	Cache content at 600+ PoPs	200ms→<10ms cached	S3 + ALB origins, WAF attached
Route 53 DNS	Global DNS with 100% SLA	<5ms DNS resolution	Latency-based routing to ALB
Global Accelerator	TCP/UDP via AWS private backbone	30-50ms reduction	Checkout endpoint only
Lambda@Edge	JS code at edge PoPs	Adds 1-5ms at edge	A/B testing, auth header signing
CloudFront Functions	Ultra-lightweight JS at edge	Sub-1ms overhead	URL normalization, header rewrite

CloudFront Cache Behavior — ShopFlow

```
Path Pattern Origin Cache TTL Compress
/static/* S3 365 days gzip+brotli
/images/* S3 30 days gzip
```

```
/api/v1/products* ALB 60 seconds gzip  
/api/v1/orders* ALB 0 (no cache) gzip  
/* (default) S3 (SPA) 0 for HTML gzip
```

■ Always use Cache-Control: max-age=31536000,immutable for content-addressed JS/CSS files. HTML files should be max-age=0 so users always get the latest React app.

AWS pricing has compute, storage, transfer, and request dimensions. Understanding all four prevents surprises on the monthly bill.

Pricing Model	Discount vs OD	Commitment	Interruptible	Best Use
On-Demand	0%	None	No	Dev, spiky, new workloads
1-Year Reserved	40%	1yr, instance family	No	Steady baseline (RDS)
3-Year Reserved	60%	3yr, instance family	No	Long-term stable DBs
Compute Savings Plan	66%	1-3yr, \$/hr commit	No	Fargate + Lambda
EC2 Savings Plan	72%	1-3yr, instance family	No	EC2-heavy workloads
Spot Instances	90%	None, 2-min notice	Yes	Batch, stateless jobs
Free Tier	100% on limits	12-month trial	No	Dev/learning

ShopFlow Cost Breakdown — 1M MAU Production

AWS Service	Configuration	Monthly Cost	Optimization
ECS Fargate	12 tasks (2vCPU/4GB), Compute SP	\$420	Scale to 4 tasks off-peak: -\$280
Aurora PostgreSQL	db.r6g.large, Multi-AZ, 1yr Reserved	\$280	Read replica saves 40% on reads
ElastiCache Redis	cache.r6g.large x2, 1yr Reserved	\$200	Cluster mode off for simplicity
OpenSearch	m6g.large x3 across 3 AZs	\$220	Cold storage for old indices: -\$80
S3 + CloudFront	500GB + 10TB transfer via CF	\$122	Lifecycle to Glacier IA: -\$40
API Gateway	50M REST calls/month	\$88	Cache responses: -\$30
SQS + SNS	1B messages/month	\$40	Standard queue over FIFO: -\$15
Lambda	100M invocations, 256MB, 500ms avg	\$35	ARM (Graviton2): -\$7
Route 53	1 hosted zone, 10M queries	\$15	Minimal
CloudWatch	Logs 50GB, 200 metrics, 50 alarms	\$85	Log retention 30 days: -\$30
CloudTrail	All regions, S3 archive	\$25	Minimal
TOTAL		~\$1,530/mo	With optimizations: ~\$1,050/mo

■ Enable AWS Cost Anomaly Detection (free) — it learns your spending patterns and alerts you when something spikes unexpectedly. Saved ShopFlow \$2,400 when a dev left Elasticsearch in prod accidentally.

Three interfaces cover all AWS operations: Console for human interaction, CLI for scripting and CI/CD, SDK for application code. ShopFlow uses all three.

AWS CLI v2 — Daily ShopFlow Operations

```
# Authenticate with SSO
aws sso login --profile shopflow-dev

# Force new ECS deployment (rolling update)
aws ecs update-service --cluster shopflow-prod \
  --service product-svc --force-new-deployment \
  --profile shopflow-prod

# Sync React build to S3 + invalidate CloudFront
aws s3 sync ./frontend/build s3://shopflow-static/ \
```

```
--cache-control "max-age=31536000,immutable" \  
--exclude "*.html" --profile shopflow-prod  
aws cloudfront create-invalidation \  
--distribution-id E1XXXXX --paths "/*.html"  
# Tail application logs in real-time  
aws logs tail /ecs/shopflow/product-svc --follow \  
--filter-pattern "ERROR" --profile shopflow-prod  
# Scale ECS service manually  
aws ecs update-service --cluster shopflow-prod \  
--service product-svc --desired-count 8
```

Java SDK v2 — Application Integration

```
// Auto-configures from ECS task role (no credentials needed)  
S3Client s3 = S3Client.builder()  
    .region(Region.US_EAST_1)  
    .build();  
SqsClient sqs = SqsClient.builder()  
    .region(Region.US_EAST_1)  
    .build();  
// Send order event to SQS  
sqs.sendMessage(SendMessageRequest.builder()  
    .queueUrl(orderQueueUrl)  
    .messageBody(objectMapper.writeValueAsString(orderEvent))  
    .messageGroupId(order.getUserId()) // FIFO queue  
    .build());
```

■ SDK v2 uses DefaultCredentialsProvider chain: env vars → system properties → ~/.aws → ECS task role → EC2 instance profile. In ECS, the task role is automatically used — never inject credentials.

AWS and customers share security responsibilities. The boundary varies by service type — IaaS (EC2) gives you more control and more responsibility. PaaS/SaaS (Lambda, RDS) means AWS manages more.

Layer	AWS Manages	You Manage
Physical	Data center security, hardware lifecycle, disposal	Nothing
Network infra	Global backbone, DDoS scrubbing, private fiber	VPC, Security Groups, NACLs, routing
Compute hypervisor	EC2 Nitro hypervisor, VM isolation, host patching	Guest OS on EC2, container image CVEs
Managed services	RDS engine patches, Redis upgrades, Lambda runtime	Params, config, IAM permissions, data
Storage encryption	Hardware encryption capability on all storage	Enable SSE-KMS, set bucket policies
Identity	IAM service availability, hardware MFA tokens	Policies, MFA enforcement, key rotation
DDoS (Layer 3/4)	Shield Standard protects all services automatically	Shield Advanced for L7, WAF rules
Compliance certs	SOC1/2/3, ISO27001, PCI-DSS infrastructure certs	App-level PCI scope, GDPR, HIPAA controls

✓ Enable MFA on root account and all privileged IAM users

✓ Use IAM roles (not user access keys) for all workloads on AWS

✓ Enable AWS Config to continuously audit resource configurations

✓ Enable GuardDuty for threat detection across all accounts

■ Public S3 buckets, wide-open Security Groups, and root API keys = instant breach

■ Compliance cert (AWS PCI-DSS) covers AWS infra only — your app needs its own PCI audit

Single-account setups fail at scale. AWS Organizations provides the governance framework for multi-account environments.

AWS Organization — ShopFlow Inc.

Management Account — billing, SSO, SCP policies only

Security OU — centralized security tooling

Security Account: GuardDuty aggregator, SecurityHub, Config

Infrastructure OU — shared platform services

Shared Services: ECR, CodePipeline, Route53 hosted zones

Network: Transit Gateway, Direct Connect, VPN

Workloads OU — application environments

Production: live traffic, break-glass access only

Staging: mirrors prod at 25%, QA + integration tests

Development: engineer sandboxes, auto-nuke after 7 days idle

Service Control Policies (SCPs) Examples


```
// SCP 1: Restrict to approved regions only
{"Effect":"Deny","Action":"*","Resource":"*",
"Condition":{"StringNotIn":{"aws:RequestedRegion":
["us-east-1","eu-west-1"]}}}

// SCP 2: Deny disabling security services
{"Effect":"Deny","Action":["cloudtrail:DeleteTrail",
"guardduty:DeleteDetector","config:DeleteConfigRule"],
"Resource":"*"}
```

■ Apply SCPs at the OU level. Test in a sandbox account first — SCPs can lock out entire OUs if misconfigured. Use "Deny" list (blacklist) approach for flexibility.

ShopFlow Production — us-east-1 (3 AZs)

DNS & Edge (Global)

Route 53: latency routing → CloudFront distribution

CloudFront: WAF + Shield Advanced, gzip/brotli, HTTP/2

ACM: *.shopflow.com wildcard TLS (auto-renewed)

VPC 10.0.0.0/16

Public Subnets /24 x3: ALB, NAT Gateways only

Private App Subnets /24 x3: ECS Fargate, Lambda in VPC

Private Data Subnets /24 x3: RDS, Redis, OpenSearch

Compute (Private App Subnets)

ECS Fargate: 5 services, Target Tracking ASP (CPU 60%)

Lambda: 8 functions — thumbnail, email, cron, webhooks

API Gateway: REST /v1/* → ALB; WS /live/* → Lambda

Data (Private Data Subnets)

Aurora PostgreSQL: Writer 1a, Reader 1b, failover to 1c

DynamoDB Global Tables: us-east-1 + eu-west-1 replicas

ElastiCache Redis 7: Primary 1a + Replica 1b

OpenSearch 2.11: 3 data nodes across 3 AZs

Messaging (Cross-AZ)

SQS: order-queue.fifo, email-queue (std), dlq for each

SNS: order-events → email + SMS + push endpoints

Kinesis: clickstream (3 shards, 24hr retention)

Storage & Observability

S3: shopflow-images, shopflow-static, shopflow-exports

CloudWatch: logs/metrics/alarms + X-Ray traces + CloudTrail

Pillar	Core Question	ShopFlow Implementation	Maturity
Operational Excellence	How do you run & improve?	CDK + CodePipeline + runbooks + CloudWatch	High

Pillar	Core Question	ShopFlow Implementation	Maturity
Security	How do you protect?	KMS + WAF + GuardDuty + least privilege IAM	High
Reliability	How do you recover?	Multi-AZ + SQS DLQ + health checks + chaos	Medium
Performance	How do you use resources?	ElastiCache + Graviton3 + Lambda + profiling	Medium
Cost	How do you optimize?	Savings Plans + Spot batch + S3 lifecycle	Medium
Sustainability	What is your carbon footprint?	Graviton + Fargate + scale-to-zero Lambda	Low (improving)

■ Run the Well-Architected Tool review quarterly. ShopFlow targets 0 High-Risk Issues (HRIs) before each quarterly release. HRI examples: no MFA on root, no CloudTrail, public RDS.

01-11

Chapter Roadmap — All 20 Chapters

Ch	Topic	Key AWS Services	ShopFlow Feature Built
02	IAM & Security	IAM, STS, KMS, Secrets Manager	Role-based access, API key management
03	VPC & Networking	VPC, Subnets, SG, NACL, NAT	Production network isolation
04	EC2 & Auto Scaling	EC2, ASG, Launch Templates, AMI	Compute baseline, Black Friday scaling
05	S3 & CloudFront	S3, CloudFront, ACM, Lifecycle	Product images, React SPA, CDN
06	RDS & Aurora	RDS, Aurora, Read Replicas, Proxy	Orders/users DB, zero-downtime failover
07	DynamoDB	DynamoDB, DAX, GSI/LSI, Streams	Cart, sessions, inventory counters
08	ElastiCache	Redis, Cluster, ElastiCache Serverless	Product cache, rate limiting
09	Lambda & API GW	Lambda, API GW, SAM, Layers	Serverless thumbnails, checkout hooks
10	SQS, SNS, EventBridge	SQS, SNS, EventBridge, Step Functions	Order pipeline, notification fan-out
11	ECS & EKS	ECS Fargate, EKS, ECR, Compose	Microservice deployment + autoscaling
12	OpenSearch	OpenSearch, Kibana, Index aliases	Product search, analytics dashboard
13	Route 53 & ALB	Route 53, ALB, NLB, Target Groups	DNS, HTTPS, weighted canary deploy
14	Observability	CloudWatch, X-Ray, CloudTrail	Full-stack monitoring, audit trail
15	CloudFormation & CDK	CloudFormation, CDK, StackSets	All ShopFlow infra as code
16	CI/CD Pipeline	CodePipeline, CodeBuild, CodeDeploy	Blue-green automated deployments
17	Cognito & WAF	Cognito, WAF, Shield, ACM	User auth, OAuth 2.0, bot protection
18	Kinesis & Analytics	Kinesis, Firehose, Athena, QuickSight	Real-time sales + clickstream analytics
19	Cost Optimization	Cost Explorer, Budgets, CUR	Reduce ShopFlow bill by 40%
20	HA & Disaster Recovery	Route 53 failover, Global Tables	Multi-region active-passive failover

Follow these steps to provision a complete ShopFlow dev environment on AWS. The dev stack costs approximately \$3-5/day and can be torn down instantly with `cdk destroy`.

Step 1: Install Tools

```
# AWS CLI v2
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o awscliv2.zip
unzip awscliv2.zip && sudo ./aws/install
# Node.js 20 + AWS CDK v2
nvm install 20 && npm install -g aws-cdk
# Java 21 + Maven (via SDKMAN)
sdk install java 21.0.2-graalce
sdk install maven 3.9.6
# Docker Desktop (for local container builds)
# Install from https://docs.docker.com/desktop/
```

Step 2: Configure AWS SSO

```
aws configure sso
SSO session name: shopflow
SSO start URL: https://your-org.awsapps.com/start
Region: us-east-1
Account ID: 123456789012
Role: Developer
Profile name: shopflow-dev
aws sso login --profile shopflow-dev
aws sts get-caller-identity --profile shopflow-dev
```

Step 3: Clone and Deploy

```
git clone https://github.com/shopflow/shopflow-aws.git
cd shopflow-aws/infra && npm install
# Bootstrap CDK (once per account/region)
cdk bootstrap aws://123456789012/us-east-1 --profile shopflow-dev
# Deploy all stacks (~15 minutes, ~$3-5/day in dev)
cdk deploy --all --profile shopflow-dev
# To save costs when not working:
cdk destroy --all --profile shopflow-dev
```

Repository Layout

```
shopflow/
├── infra/ # CDK infrastructure (Ch15)
├── lib/
├── vpc-stack.ts # Ch03
├── data-stack.ts # Ch06-08
├── compute-stack.ts # Ch09-11
├── pipeline-stack.ts # Ch16
├── services/ # Spring Boot 3 + Java 21
├── product-svc/ order-svc/ user-svc/ notify-svc/
├── frontend/ # React 18 + TypeScript
├── .github/workflows/ # CI/CD (Ch16)
```

■ Use `cdk diff` before every deploy to preview changes. Use `--hotswap` flag for Lambda/ECS-only changes — 10x faster than full CloudFormation stack update.

Setting up cost controls is the very first thing to do in any new AWS account. AWS Budgets can alert you via email, SNS, or even trigger Lambda functions to auto-remediate runaway costs.

Recommended Budgets for ShopFlow Accounts

Account	Budget Type	Alert Threshold	Action
Development	Monthly cost	\$200 (80%), \$250 (100%)	Email dev team + Slack
Staging	Monthly cost	\$400 (80%), \$500 (100%)	Email ops team + Slack
Production	Monthly cost	\$1,800 (80%), \$2,000 (100%)	Email CTO + PagerDuty
All Accounts	Anomaly Detection	>\$50 spike vs baseline	Email + Slack + Lambda
Free Tier	Usage budget	80% of free tier limit	Email developer immediately

AWS Budget Setup via CLI

```
# Create a monthly cost budget with email alert
aws budgets create-budget \
--account-id 123456789012 \
--budget file://budget-dev.json \
--notifications-with-subscribers file://notifications.json
# budget-dev.json
{"BudgetName":"ShopFlow-Dev-Monthly",
 "BudgetLimit":{"Amount":"200","Unit":"USD"},
 "TimeUnit":"MONTHLY","BudgetType":"COST"}
# Enable AWS Cost Anomaly Detection
aws ce create-anomaly-monitor \
--anomaly-monitor MonitorType=DIMENSIONAL,
MonitorDimension=SERVICE
aws ce create-anomaly-subscription \
--anomaly-subscription Threshold=50,
Subscribers=[{Address=ops@shopflow.com,Type=EMAIL}]
```

Cost Allocation Tags — Mandatory for ShopFlow

- Activate cost allocation tags in Billing Console: Environment, Team, Service, CostCenter
- Apply tags at resource creation (CDK Tags.of(stack).add(...) applies to all nested resources)
- Use AWS Cost Explorer tag filters to break down by service team or environment
- Monthly tag compliance report from AWS Config rule: required-tags

■ Cost Anomaly Detection (free) learns your spend patterns and alerts you to spikes before they become large bills. Enable it in every account on Day 1 — it has caught runaway Elasticsearch clusters and forgotten EC2 spot fleets many times.

01-BP

Best Practices & Anti-Patterns

- ✓ Use AWS Organizations from Day 1 — at minimum: prod, dev, security accounts
- ✓ Enable Cost Budgets + Anomaly Detection before running any resources
- ✓ Deploy across 3 AZs for all production workloads from the start
- ✓ Tag every resource: Environment, Team, Service, CostCenter, ManagedBy
- ✓ Use IAM Identity Center (SSO) — never create long-lived IAM user access keys
- ✓ Enable CloudTrail, Config, and GuardDuty in every account on Day 1
- ✓ Place CloudFront in front of all user-facing content, never serve S3 directly
- ✓ Use Compute Savings Plans for Fargate/Lambda to cut costs by 66%
- Never operate from root account — create SSO admin group immediately
- Never hardcode credentials in code, Dockerfiles, or environment variables
- Never create Security Groups with 0.0.0.0/0 on DB ports (5432, 3306, 6379)
- Never deploy only to one AZ or one Region for production workloads

■ Never make RDS, ElastiCache, or OpenSearch internet-accessible directly

Concept	Key Fact
Total Regions	33 geographic regions (2024), growing quarterly
Total AZs	105, minimum 3 per Region in most regions
Edge Locations	600+ CloudFront PoPs + Route 53 resolvers
Free Tier	750h t2.micro, 5GB S3, 25GB DynamoDB, 1M Lambda/mo (12 months)
EC2 billing	Per second, minimum 60s
Lambda billing	Per 1ms duration + per invocation (\$0.20/million)
Fargate billing	Per vCPU-second + per GB-second (no idle cost)
S3 outbound (direct)	\$0.09/GB to internet (us-east-1)
S3 via CloudFront	\$0.0085/GB — 90% cheaper than direct
AZ cross-transfer	\$0.01/GB within same Region
Spot discount	Up to 90% vs On-Demand, 2-min interruption notice
Reserved max discount	72% (3-year, All Upfront, EC2 Savings Plan)
SCPs	Service Control Policies — allow/deny actions at OU/account level
Well-Architected pillars	6: OpEx, Security, Reliability, Performance, Cost, Sustainability

Top Interview Questions & Answers

Question	Answer
Region vs AZ?	Region = geographic cluster (e.g., us-east-1). AZ = isolated DC within Region (us-east-1a). AZs are independent of each other and the Region.
Shared Responsibility Model?	AWS: physical security, network infra, hypervisor, managed engine patching. You: IAM, VPC/S3, EC2, etc.
Why multiple accounts?	Blast radius isolation, separate billing, SCP enforcement, independent service limits, security best practices.
Spot vs Reserved vs On-Demand?	Spot: 90% off, batch/stateless, can be reclaimed 2-min notice. Reserved: 72% off, 3yr commit, predictable. On-Demand: full price, no commitment.
Why use CloudFront over direct S3?	Cost: CloudFront \$0.0085/GB vs S3 \$0.09/GB (10x cheaper). Performance: cached at 600+ PoPs globally.
What is AWS Organizations?	Manages multiple accounts: consolidated billing, SCPs to restrict actions per OU, account groupings.
EC2 vs Fargate vs Lambda?	EC2: full control, manage OS, cheapest at scale. Fargate: serverless containers, no OS management. Lambda: fully managed, event-driven, no server management.